

Nome do projeto:

Feira digital.

Objetivo da aplicação:

Um portal de trocas entre usuários.

problema que a aplicação procura resolver:

As trocas entre os usuários serem mais fáceis

público que poderia utilizar a aplicação:

Pessoas com o interesse em trocar itens entre si.

Tecnologias utilizadas :

- **Html** : A estrutura do site
- **CSS** : A estilização do site
- **JavaScript** : A parte Lógica do site
- **DOM** : A funcionalidade do site
- **LocalStorage** : A parte da memória do site

Nome das funcionalidades :

- **filtros por categoria** : filtra o item por categoria selecionada.
- **resumo das trocas** : resumo dos itens, se eles são disponíveis, reservados ou trocados e também mostra o total de itens.
- **descrição do item** : uma pequena ou grande descrição do seu item.
- **estado de conservação** : estado de conservação do item tipo novo, usado ou danificado.
- **nome do item** : nome do item do usuário.
- **nome do responsável** : nome do responsável pelo item adicionado.

quais informações são armazenadas :

- os itens adicionados
- a descrição do item
- a categoria do item
- os itens que estão disponíveis
- os itens que já foram reservados
- os itens que já foram trocados
- o total de itens
- o nome do usuário
- nome do item
- estado de conservação

como um item é identificado :

- pela propriedade EX: id, classe, tag name, etc

como um item é criado :

um novo **item** é criado quando o formulário é enviado,
Depois valida se os campos obrigatórios foram preenchidos.
Se estiver tudo correto, adiciona um objeto ao array

como um item é alterado :

Nesse código, um item não é alterado diretamente pelo usuário através de um formulário de edição.

A única alteração existente acontece quando o item é reservado.
Primeiro, o código pega o id armazenado no botão

como um item é removido :

um item é removido quando o botão excluir é clicado
o código identifica o item pelo o id, depois cria um array sem item,
a condição mantém todos os itens diferentes do selecionado.

assim o item com ID correspondente é retirado do array,
por fim atualiza a lista exibida na página.

como os dados permanecem disponíveis depois que a página é atualizada :

quando a página é recarregada, o javascript é executado novamente e o array volta a ficar vazio.

O localStorage ou outro banco de dados não está sendo usado, para manter os dados, seria necessário salvar e carregar os itens.

qual é a função do Local Storage no projeto :

O localStorage serve pra armazenar dados no navegador, permitindo que eles continuem disponíveis mesmo depois de atualizar ou fechar a página.

```
const formulario = document.querySelector("#formulario");
const total = document.querySelector("#total");
const disponiveis = document.querySelector("#disponiveis");
const reservados = document.querySelector("#reservados");
const trocados = document.querySelector("#trocados");
const listaResumo = document.querySelector("#lista-resumo");
const filtroCategoria = document.querySelector("#filtro-categoria");
const mensagemSucesso = document.querySelector("#mensagem-sucesso");
```

Captura dos dados

```

function atualizarResumo() {
  const categoriaSelecionada = filtroCategoria.value;
  let itensFiltrados = [];

  if (categoriaSelecionada === "") {
    itensFiltrados = itens;
  } else {
    for (let i = 0; i < itens.length; i++) {
      if (let itensFiltrados: any[] categoriaSelecionada) {
        itensFiltrados.push(itens[i]);
      }
    }
  }

  let totalItens = 0;
  let totalDisponiveis = 0;
  let totalReservados = 0;
  let totalTrocados = 0;

  for (let i = 0; i < itensFiltrados.length; i++) {
    totalItens += 1;

    if (itensFiltrados[i].disponibilidade === "sim") {
      totalDisponiveis += 1;
    } else if (itensFiltrados[i].disponibilidade === "nao") {
      totalReservados += 1;
    } else if (itensFiltrados[i].disponibilidade === "trocado") {
      totalTrocados += 1;
    }
  }

  total.textContent = "Total: " + totalItens;
  disponiveis.textContent = "Disponíveis: " + totalDisponiveis;
  reservados.textContent = "Reservados: " + totalReservados;
  trocados.textContent = "Trocados: " + totalTrocados;
}

```

esse código atualiza a página

```

listaResumo.addEventListener("click", function (event) {
  const botao = event.target.closest("button[data-action]");

  if (!botao) {
    return;
  }

  const idItem = Number(botao.dataset.id);
  const acao = botao.dataset.action;

  if (acao === "reservar") {
    for (let i = 0; i < itens.length; i++) {
      if (itens[i].id === idItem) {
        itens[i].disponibilidade = "nao";
        break;
      }
    }

    mensagemSucesso.textContent = "Item reservado, agora tá segurado!";
    mensagemSucesso.classList.remove("escondida");
  }

  if (acao === "excluir") {
    itens = itens.filter(function (item) {
      return item.id !== idItem;
    });

    mensagemSucesso.textContent = "Item saiu da lista, foi embora!";
    mensagemSucesso.classList.remove("escondida");
  }

  atualizarResumo();
});

```

essa linha de código controla os dois botões de excluir e reservar

```

formulario.addEventListener("submit", function (event) {
    event.preventDefault();

    const nomeResponsavel = document
        .getElementById("Nome do responsavel")
        .value.trim();
    const nomeItem = document.getElementById("Nome do item").value.trim();
    const categoria = document.getElementById("categoria do item").value;
    const estado = document.getElementById("estado de conservação").value;
    const disponibilidade = document.getElementById(
        "disponibilidade para troca",
    ).value;
    const descricao = document.getElementById("descricao do item").value.trim();

    if (!nomeResponsavel) {
        alert("Preencha todos os campos antes de enviar.");
        return;
    }

    if (!nomeItem) {
        alert("Preencha todos os campos antes de enviar.");
        return;
    }

    if (!categoria) {
        alert("Preencha todos os campos antes de enviar.");
        return;
    }

    if (!estado) {
        alert("Preencha todos os campos antes de enviar.");
        return;
    }

    if (!disponibilidade) {
        alert("Preencha todos os campos antes de enviar.");
        return;
    }

```

esse código controla o envio do formulário e cadastra um novo item

qual evento foi utilizado :

Foi utilizado o evento **submit** e o **click**, o evento de submit acontece quando o usuário envia o formulário, e o evento click é usado para detectar os cliques nos botões

funcionamento :

1. O usuário acessa a aplicação
2. O usuário preenche o formulário
3. O sistema valida as informações
4. O item é cadastrado
5. O item é salvo
6. Fica disponível para troca ou para ser retirado
7. Fim da aplicação

Problemas encontrados durante o desenvolvimento :

o maior problema foi na parte de salvar o item, que a função do local storage não estava funcionando corretamente, e então eu reescrevi o código, mas acho que ainda pode ser melhorado.